ABV – Indian Institute of Information Technology and Management Gwalior, Madhya Pradesh

Microcontrollers and Embedded Systems

Project Report

Data Augmentation and Implementation of Complex YOLOv4 on ZYNQ Board

Submitted By: -

1. Aniket Palarapu
2. Abhishek Singh
3. Balram Kumar
4. Tatikonda Tejesh

Under the Guidance of:

Prof. Manisha Pattanaik
Department of Electrical and Electronics
Teaching Assistant: Priya

Submission Date:

April 14, 2026

Abstract

This project presents the design and implementation of an autonomous vehicle detection system using Complex YOLOv4, a 3D object detection model, with deployment on the Xilinx ZYNQ FPGA board. The work involves building a custom dataset of Indian road vehicles, applying data augmentation techniques to increase dataset diversity, generating KITTI-format 3D labels using Velodyne LiDAR data, and training the Complex YOLOv4 model for real-time 3D object detection from Bird's Eye View (BEV) representations.

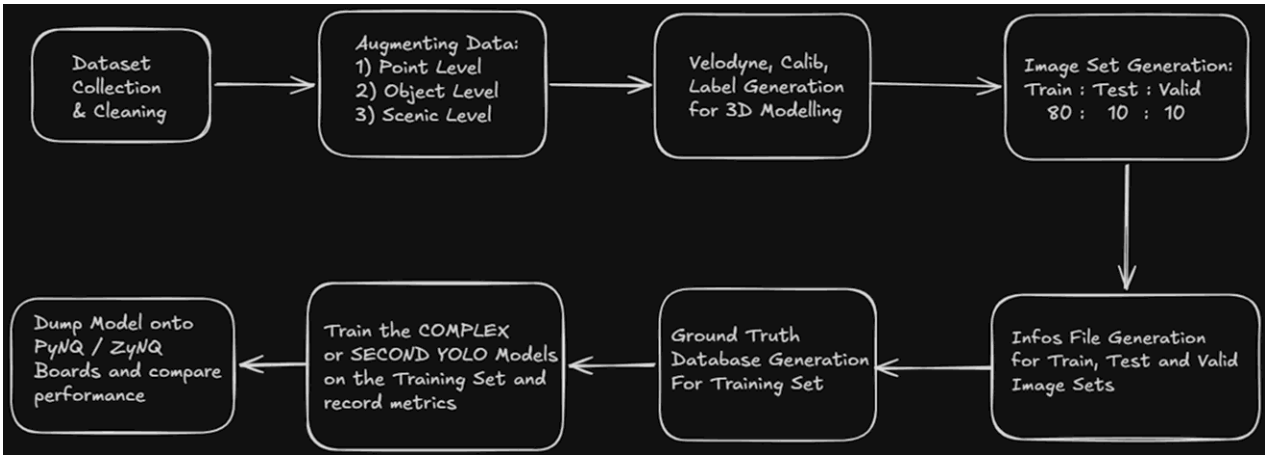
A complete pipeline from dataset collection and cleaning, through augmentation and KITTI-format generation, to model training and FPGA deployment via the Vitis AI framework is developed and documented. The system converts the trained PyTorch model to ONNX format, quantizes it from FP32 to INT8 precision, compiles it into an xmodel using the Vitis AI compiler, and deploys it on the ZYNQ board using the DPU runtime. The project also documents key challenges encountered during training and the lessons learned, contributing to an honest and transparent account of the development process.

Problem Statement

- Create an autonomous vehicle detection system using Complex YOLOv4 for accurate real-time 3D object detection.
- Develop a custom dataset of different Indian vehicles such as cars, motorcycles, buses, cyclists, auto rickshaws, and heavy vehicles from real-world Indian environments.
- Apply data augmentation techniques to increase dataset size and improve model performance, robustness, and diversity.
- Deploy a stable trained model on ZYNQ boards and monitor system performance in real-time applications.

Methodology

The project follows a structured eight-step pipeline from raw data collection to FPGA deployment. Each stage builds upon the previous one, forming a complete end-to-end autonomous vehicle detection workflow.



Step 1: Dataset Collection

- Collected 2,400 images of various vehicles including cars, buses, motorcycles, and heavy vehicles.
- Captured images from Indian roads and highways under different positions, angles, and viewpoints to ensure realistic data.
- Created a diverse dataset representing real-world traffic conditions in the Indian environment.

Step 2: Dataset Cleaning

- Cleaned the collected dataset to improve overall data quality and consistency.
- Removed redundant, noisy, similar, and unclear images that could negatively affect model performance.
- Removed images with inconsistent aspect ratios.
- Reduced the dataset size from 2,400 to 1,000 high-quality images for efficient training and accurate detection.

Step 3: Data Augmentation

Post dataset cleaning, three different augmentation techniques were applied to reduce bias and increase dataset diversity:

- Scene Level Augmentation: Modifies the overall environment of the image, such as changing brightness, weather conditions, or background to simulate different real-world scenarios.
- Point Level Augmentation: Alters individual pixel values or small regions in the image, such as adding noise or adjusting contrast to improve model robustness.
- Object Level Augmentation: Transforms specific objects in the image, such as rotating, scaling, or flipping vehicles to create variations in object appearance.



Augmentation results:

- Original Dataset (after cleaning) = 1,118 images
- After Augmentation (1,118 × 3) = 3,354 images

Step 4: Velodyne, Calibration and Label Generation

Each sample in the dataset requires three corresponding files in KITTI format for 3D detection training:

- Velodyne (.bin): Contains 3D point cloud data representing real-world object geometry and spatial information.
- Calibration (.txt): Defines the transformation parameters between camera and LiDAR sensors for accurate coordinate mapping.
- Labels (.txt): Stores annotated 3D bounding boxes in KITTI format for training and evaluation of detection models.

Procedure:

1. Collected synchronized image and LiDAR data from the dataset for processing.
2. Generated object labels using YOLO-based detection models to identify vehicles in the images.
3. Converted detection outputs into KITTI 3D format suitable for 3D object detection training.

4. Verified alignment between image, LiDAR, and label data to ensure consistency and accuracy.

Step 5: Image Set Generation

After Velodyne, Calibration and Label Generation, the entire dataset is divided into three sets with the proportion of 80:10:10.

Image Set	Original Dataset	Augmented Dataset
Training Set (80%)	894 images	2,683 images
Validation Set (10%)	112 images	336 images
Testing Set (10%)	112 images	335 images

The Image Sets contain .txt files which store the names of the samples of the respective set, which are assigned completely randomly using shuffling.

Complex YOLOv4 Model

Background and Motivation

YOLOv4 is originally designed for 2D object detection in images. However, many real-world applications like autonomous driving require 3D understanding of the environment. Complex YOLOv4 extends YOLOv4 to perform real-time 3D object detection by processing spatial information from LiDAR sensors.

Key capabilities of Complex YOLOv4:

- Detects objects in three dimensions (3D) instead of just 2D.
- Provides information about position, size, and orientation/direction of objects.
- Maintains real-time speed, which is critical for autonomous systems.
- Suitable for self-driving cars, robotics, and smart surveillance.

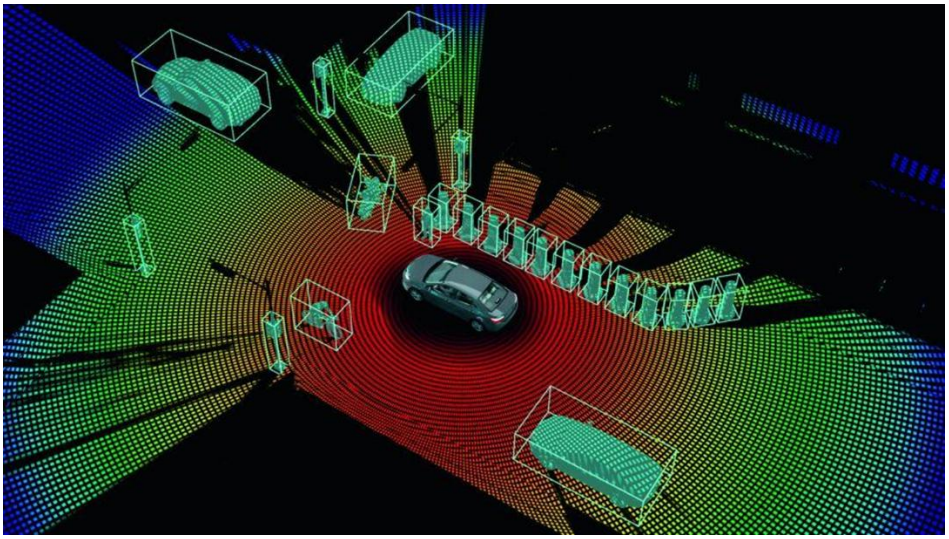
Bird's Eye View (BEV) Representation

A LiDAR sensor collects thousands of 3D points representing the environment, known as a point cloud. Processing raw 3D data is computationally expensive, so the system converts it into a Bird's Eye View (BEV) — a top-down 2D grid representation.

Reasons BEV is used:

- Simplifies complex 3D data into a 2D grid.
- Reduces computation significantly.
- Allows the use of fast CNN-based models.
- Preserves the spatial layout of objects.

The pipeline becomes: LiDAR → BEV Map → Neural Network.

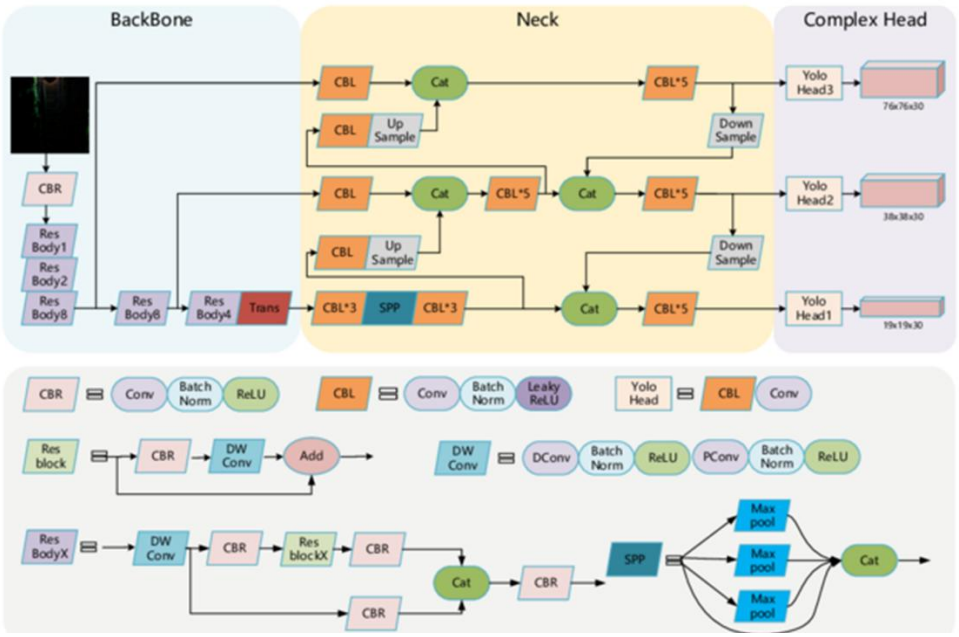


Overall Network Pipeline

The Complex YOLOv4 pipeline consists of four main stages:

- Input Layer: Accepts the BEV image converted from the LiDAR point cloud.
- Backbone CNN: Extracts multi-scale spatial features from the BEV image.
- Feature Reorganization: Fuses and reorganizes feature maps across scales.
- Detection Head (E-RPN): Predicts object location, dimensions, class, confidence, and orientation angle.

The network predicts for each detected object: Position (x, y), Dimensions (w, l), Class probability, Confidence score, and Orientation angle.



Training

Model Modifications

The Complex YOLOv4 architecture was modified to accommodate the custom dataset with 9 classes (Animal, Auto, Bicycle, Bus, Car, Motorcycle, Pedestrian, Tractor, Truck). The number of filters in the detection head was updated to 48 based on the standard relation: filters = (classes + 5) × 3.

Dataset Class Distribution

The dataset consists of 9 vehicle classes. The following table shows the distribution of annotated instances across training and validation splits, in both raw image and BEV format:

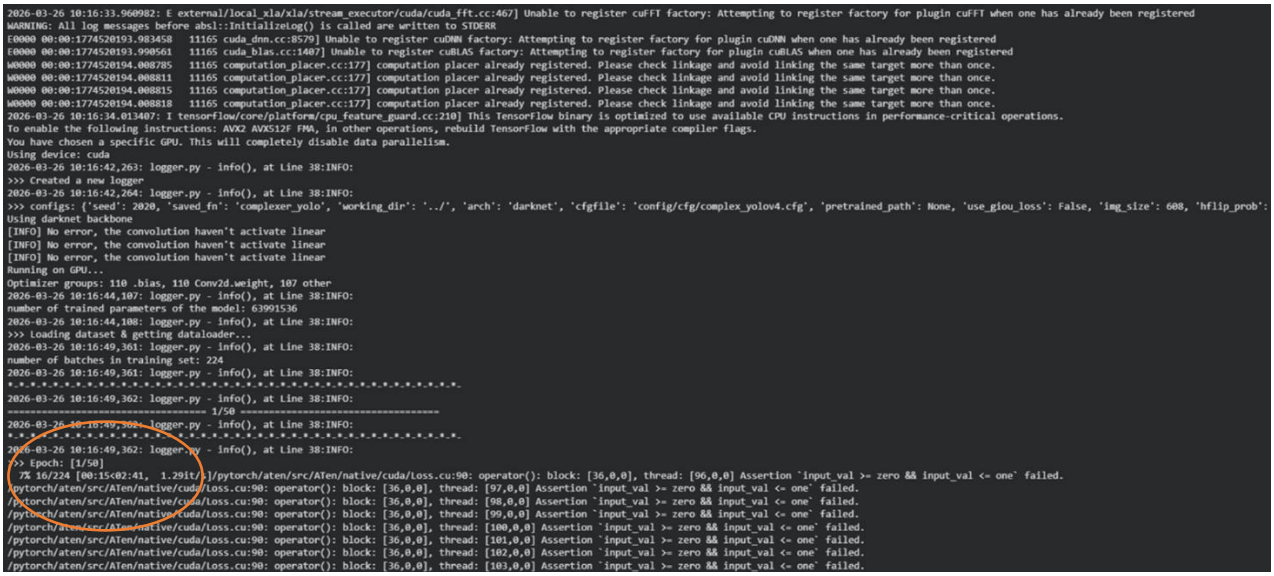
Class	ID	Train (raw)	Train (BEV)	Val (raw)	Val (BEV)
Animal	0	2	2	0	0
Auto	1	1	0	0	0
Bicycle	2	1,372	702	156	66
Bus	3	253	151	42	26
Car	4	2,554	1,429	340	198
Motorcycle	5	186	109	27	9
Pedestrian	6	792	387	101	54

Class	ID	Train (raw)	Train (BEV)	Val (raw)	Val (BEV)
Tractor	7	852	479	119	76
Truck	8	6	2	0	0

Training Challenges and Solutions

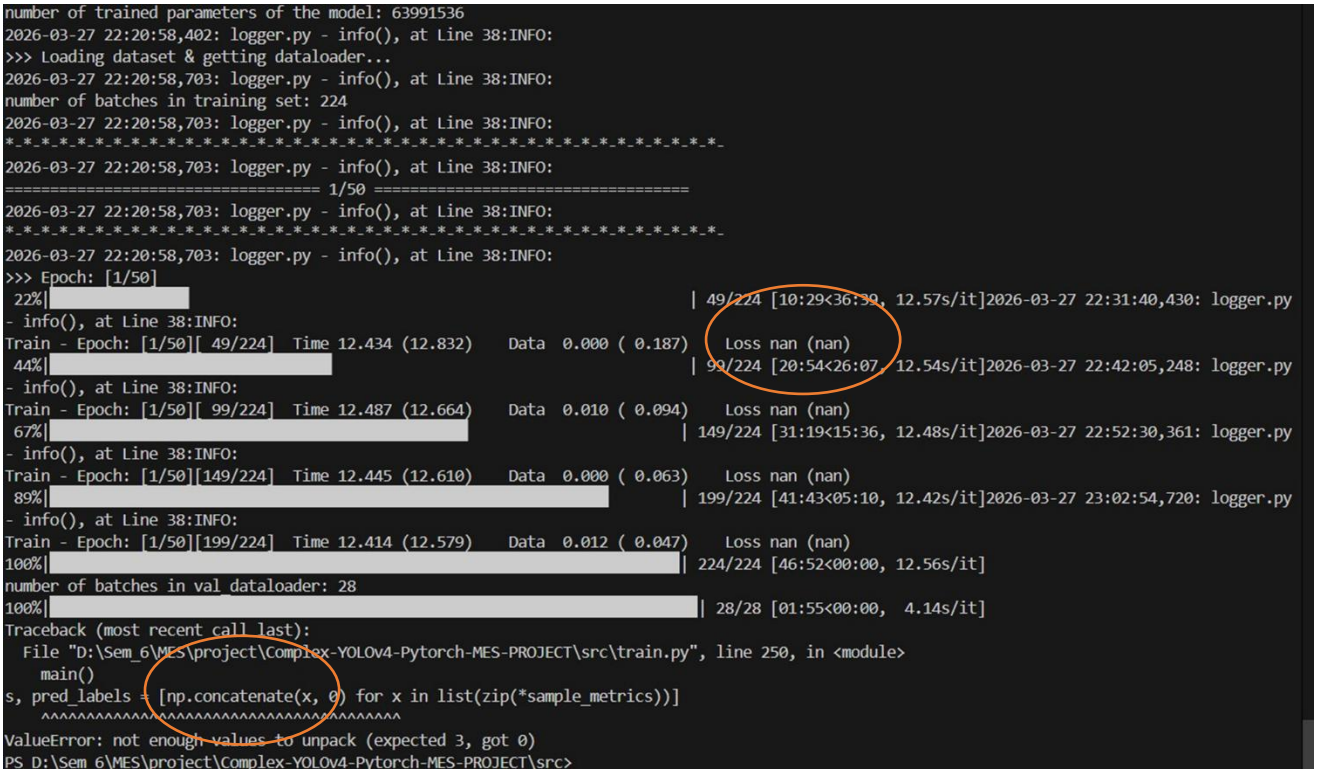
Training encountered numerous errors that provided significant learning opportunities:

- Issue: Training always crashed at the 16th batch of the first epoch.
  - Solution: Replaced BCE Loss with BCE Logistic Loss in order to keep all loss values between 0 and 1.



- Issue: Loss values reported as NaN (Not a Number) throughout Epoch 1, indicating a numerical instability in the training pipeline.
  - Observed in training logs at batches 49/224, 99/224, 149/224, 199/224 — all reporting Loss: nan.
- Issue: ValueError at end of epoch: ‘not enough values to unpack (expected 3, got 0)’ when computing sample metrics.
  - This was caused by the validation dataset returning zero predictions, resulting in empty metric arrays.

Training was run for 50 epochs with 63,991,536 trainable parameters and 224 batches per epoch.



Results and Discussion

Evaluation on Custom Dataset

The trained Complex YOLOv4 model was evaluated on the custom validation set using the evaluate.py script with the following configuration:

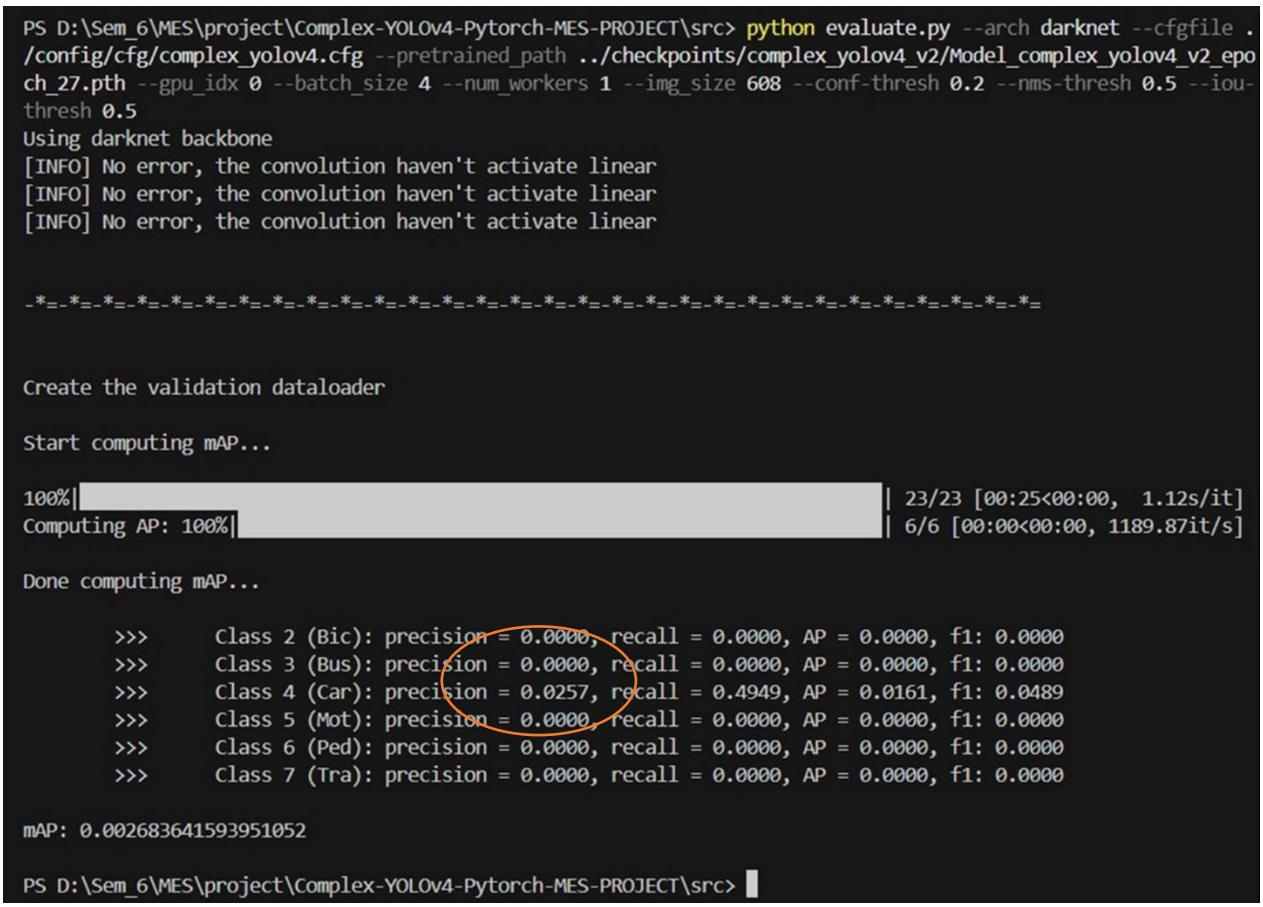
- Architecture: Darknet backbone
- Config file: complex_yolov4.cfg
- Image size: 608
- Confidence threshold: 0.2
- NMS threshold: 0.5
- IoU threshold: 0.5
- Batch size: 4

Per-class evaluation results:

Class	Precision	Recall	AP	F1 Score
Bicycle (Class 2)	0.0000	0.0000	0.0000	0.0000
Bus (Class 3)	0.0000	0.0000	0.0000	0.0000
Car (Class 4)	0.0257	0.4949	0.0161	0.0489

Class	Precision	Recall	AP	F1 Score
Motorcycle (Class 5)	0.0000	0.0000	0.0000	0.0000
Pedestrian (Class 6)	0.0000	0.0000	0.0000	0.0000
Tractor (Class 7)	0.0000	0.0000	0.0000	0.0000

Overall mAP on custom dataset: 0.00268 (approximately 0.27%)



Reasons for Unsatisfactory Results

The low mAP on the custom dataset is attributed to three primary root causes identified during analysis:

- Incorrect Label Creation: The original label generation process was flawed, which did not address the severe class imbalance in the dataset (e.g., Car: 2,554 vs Animal: 2, Auto: 1).
- Incorrect Calibration Data: The calibration matrix was set as an Identity Matrix, meaning the 2D image coordinates and 3D point cloud coordinates were treated as identical. This significantly reduced the accuracy of the BEV projection and detection precision.

- Insufficient Training Epochs: The model was trained for far fewer epochs than required. The recommended range for convergence on this architecture is approximately 300 epochs, but time constraints limited the training run.

Final Decision: KITTI-Pretrained Model for ZYNQ Deployment

Given the unsatisfactory results on the custom dataset, the team made the decision to use a Complex YOLOv4 model pre-trained on the original KITTI dataset with 3 classes (Car, Pedestrian, Cyclist) achieving 93% accuracy for deployment onto the ZYNQ board. This ensured a stable and functional demonstration of the full FPGA deployment pipeline.

ZYNQ Board Implementation

The ZYNQ board deployment pipeline consists of three main stages:

Stage 1: Complex YOLO Model Design and Training

- Designed and trained the Complex YOLOv4 architecture in PyTorch.
- Optimized model performance and saved the trained weights as a .pth checkpoint file.

Stage 2: FPGA Accelerator Synthesis using Vitis AI

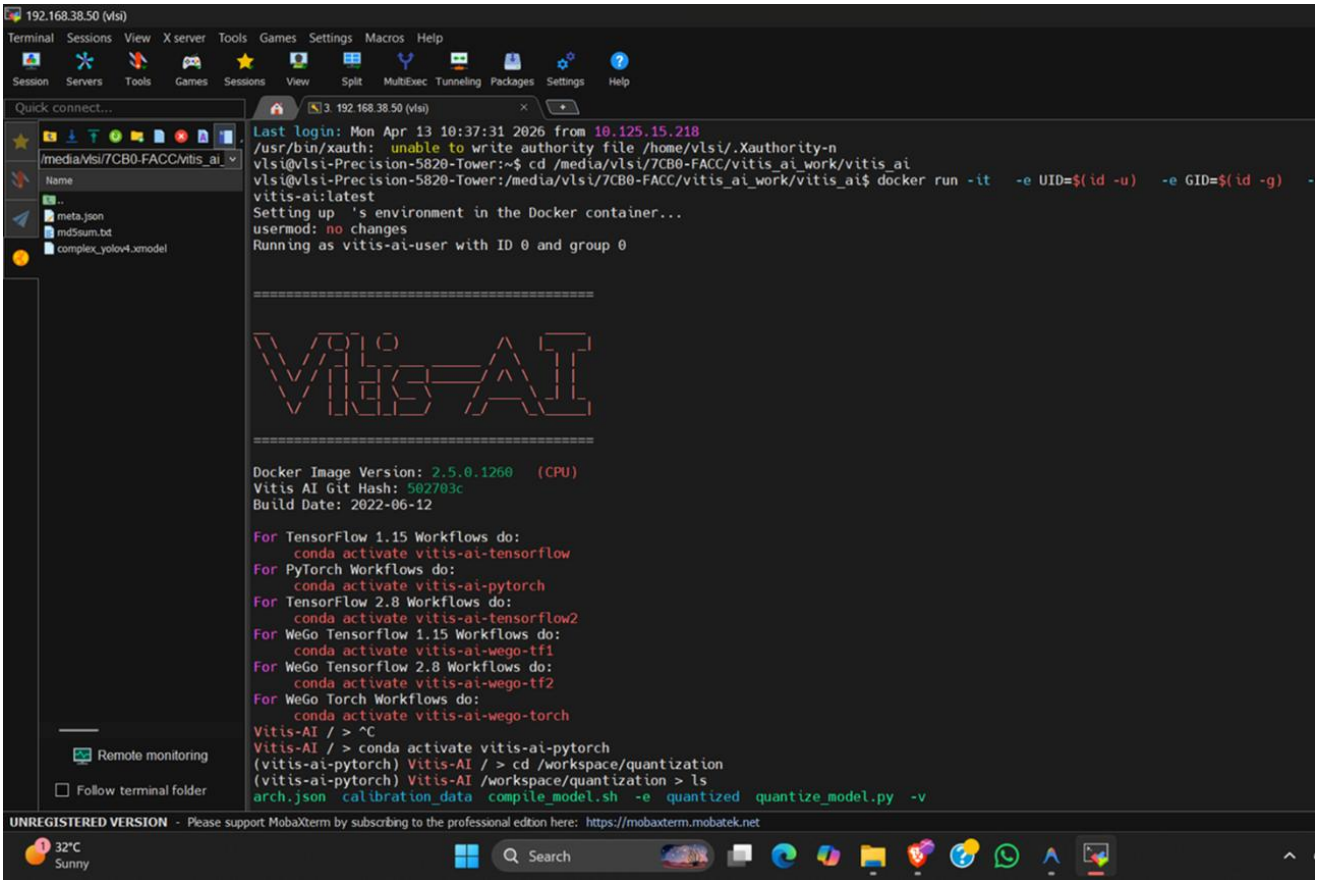
The trained model is converted and compiled for FPGA deployment through the following quantization pipeline:

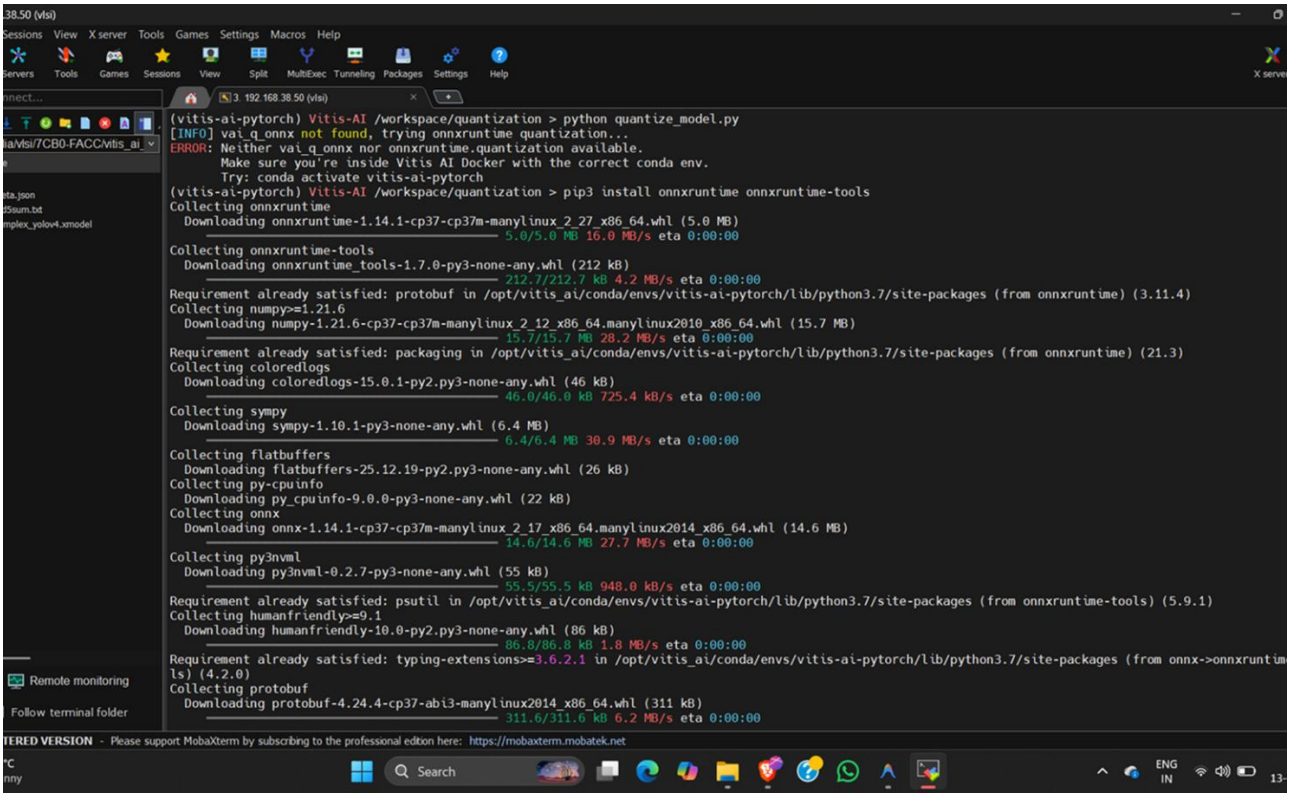
- Export trained PyTorch model to ONNX format (model.onnx).
- Quantize the model from FP32 to INT8 precision using Xilinx Vitis AI Docker (Vitis AI v2.5.0.1260, CPU).
- Run quantize_model.py within the vitis-ai-pytorch conda environment.
- Compile the quantized model using the Vitis AI Compiler to generate model.xmodel.

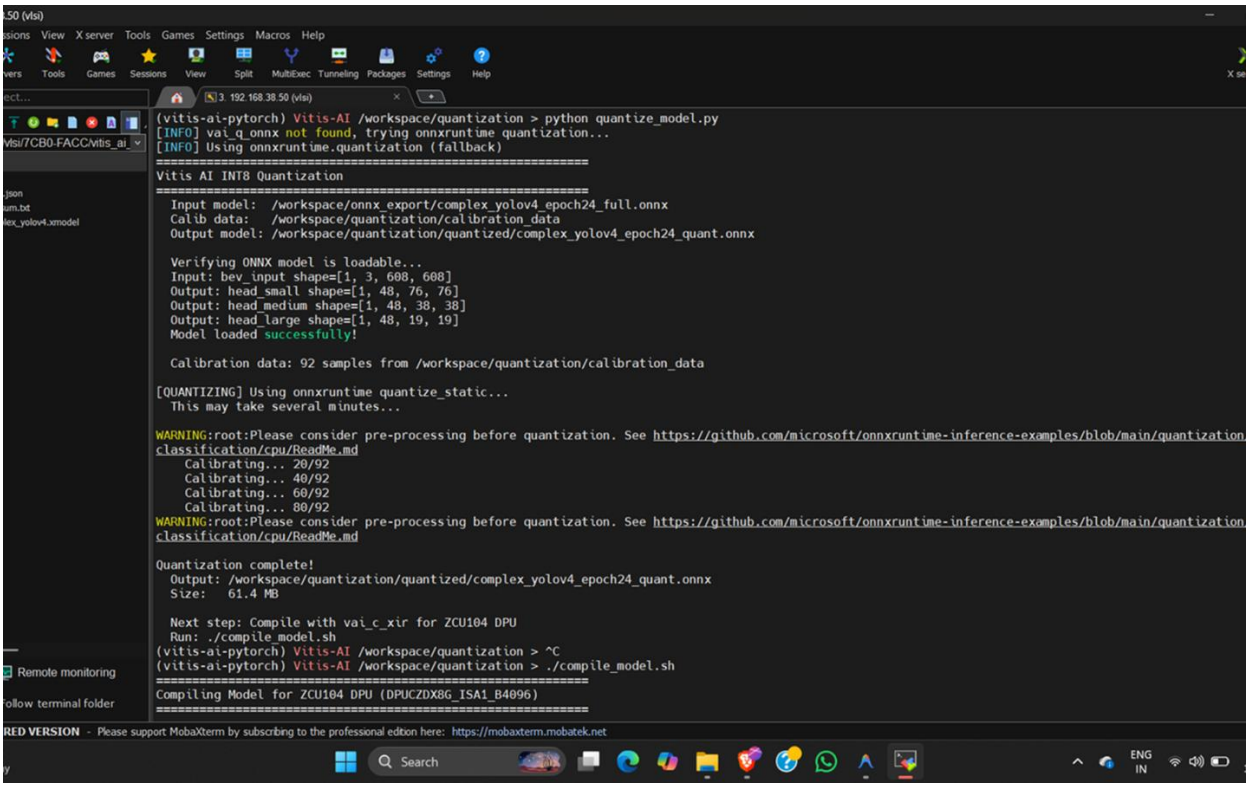
The Vitis AI Docker environment was set up on the host machine (IP: 192.168.38.50), running inside a container with the vitis-ai-pytorch conda environment. The quantization workspace contained: arch.json, calibration data, compile_model.sh, quantize_model.py, and the quantized output directory.

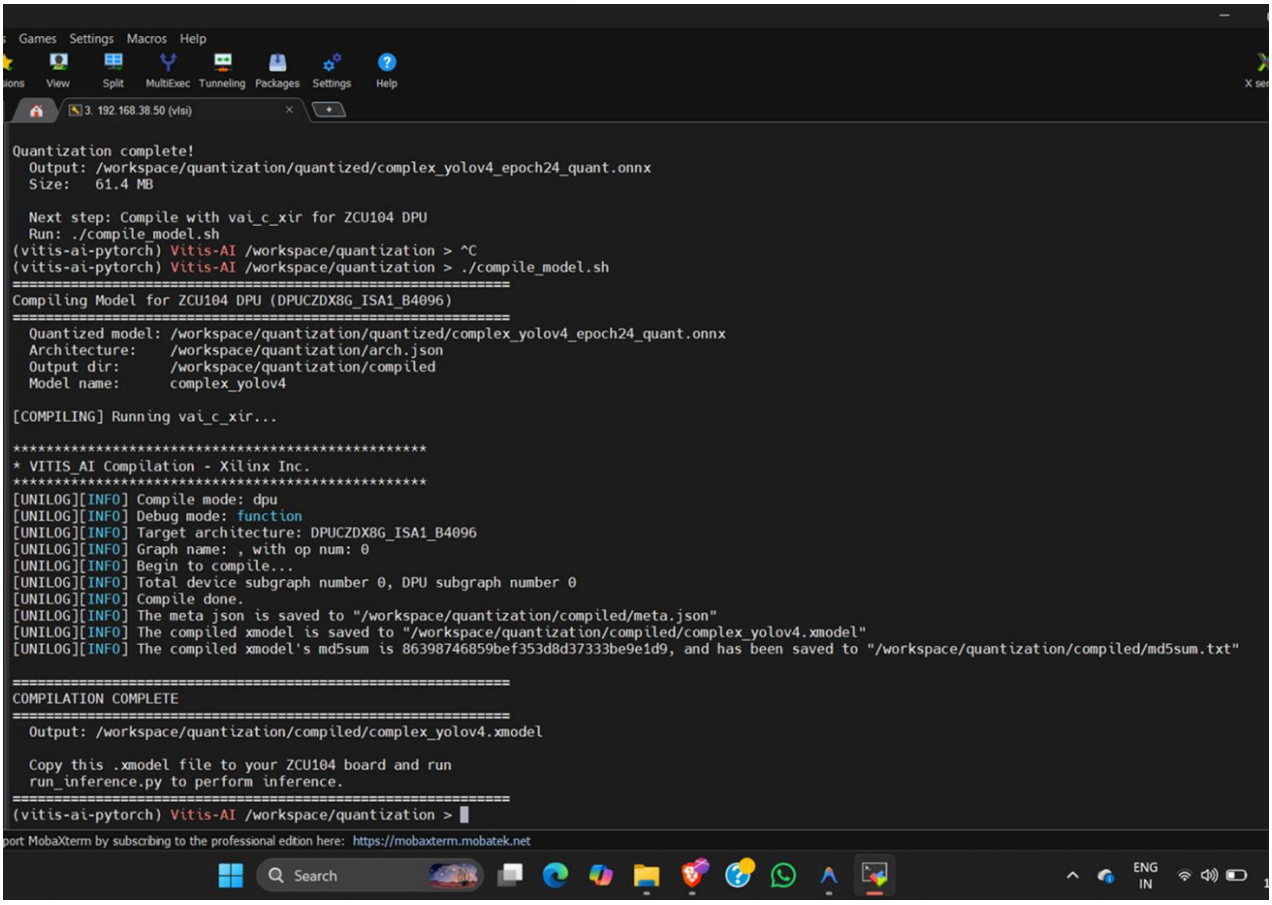
Stage 3: Zynq System Setup and Inference

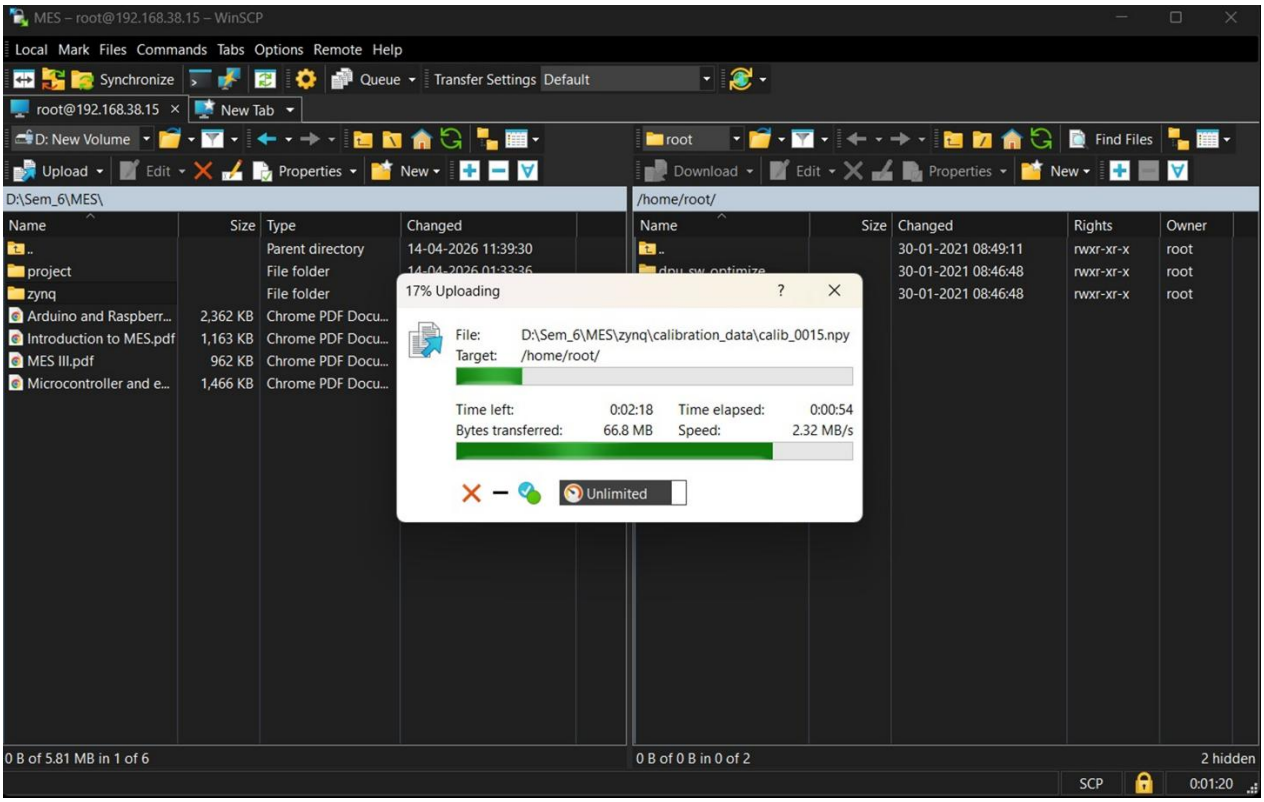
- Prepared PetaLinux image for the ZYNQ board.
- Flashed the downloaded PetaLinux image to a 16 GB (or larger) SD card.
- Connected to the board and communicated using PuTTY software over serial/SSH.
- Transferred the compiled xmodel to the board using WinSCP software.
- Loaded the DPU Runtime on the ZYNQ board.
- Executed inference on the board and obtained output predictions with bounding boxes.


192.168.38.50 (vlsi)
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help
Quick connect...
3. 192.168.38.50 (vlsi)
/media/vlsi/7CB0-FACC/vitis_ai
Name
meta.json
md5sum.txt
complex_yolov4.xmodel
Remote monitoring
Follow terminal folder
Last login: Mon Apr 13 10:37:31 2026 from 10.125.15.218
/usr/bin/xauth: unable to write authority file /home/vlsi/.Xauthority-n
vlsi@vlsi-Precision-5820-Tower:~$ cd /media/vlsi/7CB0-FACC/vitis_ai_work/vitis_ai
vlsi@vlsi-Precision-5820-Tower:/media/vlsi/7CB0-FACC/vitis_ai_work/vitis_ai$ docker run -it -e UID=$(id -u) -e GID=$(id -g)
vitis-ai:latest
Setting up 's environment in the Docker container...
usermod: no changes
Running as vitis-ai-user with ID 0 and group 0
Docker Image Version: 2.5.0.1260 (CPU)
Vitis AI Git Hash: 502703c
Build Date: 2022-06-12
For TensorFlow 1.15 Workflows do:
conda activate vitis-ai-tensorflow
For PyTorch Workflows do:
conda activate vitis-ai-pytorch
For TensorFlow 2.8 Workflows do:
conda activate vitis-ai-tensorflow2
For WeGo Tensorflow 1.15 Workflows do:
conda activate vitis-ai-wego-tf1
For WeGo Tensorflow 2.8 Workflows do:
conda activate vitis-ai-wego-tf2
For WeGo Torch Workflows do:
conda activate vitis-ai-wego-torch
Vitis-AI / > ^C
Vitis-AI / > conda activate vitis-ai-pytorch
(vitis-ai-pytorch) Vitis-AI / > cd /workspace/quantization
(vitis-ai-pytorch) Vitis-AI /workspace/quantization > ls
arch.json calibration_data compile_model.sh -e quantized quantize_model.py -v
UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net
32°C
Sunny
Search


Sessions View X server Tools Games Settings Macros Help
Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help
3. 192.168.38.50 (vlsi)
(vitis-ai-pytorch) Vitis-AI /workspace/quantization > python quantize_model.py
[INFO] vai_q_onnx not found, trying onnxruntime quantization...
ERROR: Neither vai_q_onnx nor onnxruntime.quantization available.
Make sure you're inside Vitis AI Docker with the correct conda env.
Try: conda activate vitis-ai-pytorch
(vitis-ai-pytorch) Vitis-AI /workspace/quantization > pip3 install onnxruntime onnxruntime-tools
Collecting onnxruntime
Downloading onnxruntime-1.14.1-cp37-cp37m-manylinux_2_27_x86_64.whl (5.0 MB)
5.0/5.0 MB 16.0 MB/s eta 0:00:00
Collecting onnxruntime-tools
Downloading onnxruntime_tools-1.7.0-py3-none-any.whl (212 kB)
212.7/212.7 kB 4.2 MB/s eta 0:00:00
Requirement already satisfied: protobuf in /opt/vitis_ai/conda/envs/vitis-ai-pytorch/lib/python3.7/site-packages (from onnxruntime) (3.11.4)
Collecting numpy>=1.21.6
Downloading numpy-1.21.6-cp37-cp37m-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (15.7 MB)
15.7/15.7 MB 28.2 MB/s eta 0:00:00
Requirement already satisfied: packaging in /opt/vitis_ai/conda/envs/vitis-ai-pytorch/lib/python3.7/site-packages (from onnxruntime) (21.3)
Collecting coloredlogs
Downloading coloredlogs-15.0.1-py2.py3-none-any.whl (46 kB)
46.0/46.0 kB 725.4 kB/s eta 0:00:00
Collecting sympy
Downloading sympy-1.10.1-py3-none-any.whl (6.4 MB)
6.4/6.4 MB 30.9 MB/s eta 0:00:00
Collecting flatbuffers
Downloading flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Collecting py-cpuinfo
Downloading py_cpuinfo-9.0.0-py3-none-any.whl (22 kB)
Collecting onnx
Downloading onnx-1.14.1-cp37-cp37m-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (14.6 MB)
14.6/14.6 MB 27.7 MB/s eta 0:00:00
Collecting py3nvml
Downloading py3nvml-0.2.7-py3-none-any.whl (55 kB)
55.5/55.5 kB 948.0 kB/s eta 0:00:00
Requirement already satisfied: psutil in /opt/vitis_ai/conda/envs/vitis-ai-pytorch/lib/python3.7/site-packages (from onnxruntime-tools) (5.9.1)
Collecting humanfriendly>=9.1
Downloading humanfriendly-10.0-py2.py3-none-any.whl (86 kB)
86.8/86.8 kB 1.8 MB/s eta 0:00:00
Requirement already satisfied: typing-extensions>=3.6.2.1 in /opt/vitis_ai/conda/envs/vitis-ai-pytorch/lib/python3.7/site-packages (from onnx->onnxruntime-tools) (4.2.0)
Collecting protobuf
Downloading protobuf-4.24.4-cp37-abi3-manylinux2014_x86_64.whl (311 kB)
311.6/311.6 kB 6.2 MB/s eta 0:00:00
Remote monitoring
Follow terminal folder
Search
ENG
IN


(vitis-ai-pytorch) Vitis-AI /workspace/quantization > python quantize_model.py
[INFO] vai_q_onnx not found, trying onnxruntime quantization...
[INFO] Using onnxruntime.quantization (fallback)
Vitis AI INT8 Quantization
Input model: /workspace/onnx_export/complex_yolov4_epoch24_full.onnx
Calib data: /workspace/quantization/calibration_data
Output model: /workspace/quantization/quantized/complex_yolov4_epoch24_quant.onnx
Verifying ONNX model is loadable...
Input: bev_input shape=[1, 3, 608, 608]
Output: head_small shape=[1, 48, 76, 76]
Output: head_medium shape=[1, 48, 38, 38]
Output: head_large shape=[1, 48, 19, 19]
Model loaded successfully!
Calibration data: 92 samples from /workspace/quantization/calibration_data
[QUANTIZING] Using onnxruntime quantize_static...
This may take several minutes...
WARNING:root:Please consider pre-processing before quantization. See https://github.com/microsoft/onnxruntime-inference-examples/blob/main/quantization/image_classification/cpu/ReadMe.md
Calibrating... 20/92
Calibrating... 40/92
Calibrating... 60/92
Calibrating... 80/92
WARNING:root:Please consider pre-processing before quantization. See https://github.com/microsoft/onnxruntime-inference-examples/blob/main/quantization/image_classification/cpu/ReadMe.md
Quantization complete!
Output: /workspace/quantization/quantized/complex_yolov4_epoch24_quant.onnx
Size: 61.4 MB
Next step: Compile with vai_c_xir for ZCU104 DPU
Run: ./compile_model.sh
(vitis-ai-pytorch) Vitis-AI /workspace/quantization > ^C
(vitis-ai-pytorch) Vitis-AI /workspace/quantization > ./compile_model.sh
Compiling Model for ZCU104 DPU (DPUCZDX8G_ISA1_B4096)
Remote monitoring
Follow terminal folder
Please support MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net
Search
ENG
IN


Quantization complete!
Output: /workspace/quantization/quantized/complex_yolov4_epoch24_quant.onnx
Size: 61.4 MB
Next step: Compile with vai_c_xir for ZCU104 DPU
Run: ./compile_model.sh
(vitis-ai-pytorch) Vitis-AI /workspace/quantization > ^C
(vitis-ai-pytorch) Vitis-AI /workspace/quantization > ./compile_model.sh
Compiling Model for ZCU104 DPU (DPUCZDX8G_ISA1_B4096)
Quantized model: /workspace/quantization/quantized/complex_yolov4_epoch24_quant.onnx
Architecture: /workspace/quantization/arch.json
Output dir: /workspace/quantization/compiled
Model name: complex_yolov4
[COMPILING] Running vai_c_xir...
* VITIS_AI Compilation - Xilinx Inc.
[UNILOG][INFO] Compile mode: dpu
[UNILOG][INFO] Debug mode: function
[UNILOG][INFO] Target architecture: DPUCZDX8G_ISA1_B4096
[UNILOG][INFO] Graph name: , with op num: 0
[UNILOG][INFO] Begin to compile...
[UNILOG][INFO] Total device subgraph number 0, DPU subgraph number 0
[UNILOG][INFO] Compile done.
[UNILOG][INFO] The meta json is saved to "/workspace/quantization/compiled/meta.json"
[UNILOG][INFO] The compiled xmodel is saved to "/workspace/quantization/compiled/complex_yolov4.xmodel"
[UNILOG][INFO] The compiled xmodel's md5sum is 86398746859bef353d8d37333be9e1d9, and has been saved to "/workspace/quantization/compiled/md5sum.txt"
COMPILATION COMPLETE
Output: /workspace/quantization/compiled/complex_yolov4.xmodel
Copy this .xmodel file to your ZCU104 board and run
run_inference.py to perform inference.
(vitis-ai-pytorch) Vitis-AI /workspace/quantization >
Search
ENG
IN



Team Contributions

The project was executed collaboratively, with responsibilities distributed as follows:

Aniket Palarapu and Abhishek Singh

- Led dataset collection from Indian road environments and supervised dataset cleaning.
- Implemented Scene Level and Point Level augmentation techniques.
- Created the Velodyne, Calib and Label for the created image dataset.
- Managed model training configuration and hyperparameter tuning.
- Contributed to documentation and report preparation.

Balram Kumar and Tatikonda Tejesh

- Implemented Object Level augmentation and integrated all three augmentation types.
- Developed the Velodyne, Calibration, and Label generation pipeline in KITTI format.
- Handled Image Set Generation (train/val/test splits).
- Set up the Vitis AI Docker environment and executed the full quantization and compilation pipeline.
- Configured the ZYNQ board (PetaLinux, PuTTY, WinSCP) and ran real-time inference on hardware.

Conclusion

This project successfully implemented a complete pipeline for autonomous vehicle detection using Complex YOLOv4, covering dataset creation, augmentation, KITTI-format label generation, model training, and FPGA deployment on the Xilinx ZYNQ board via the Vitis AI framework.

While the custom-trained model produced unsatisfactory detection results (mAP: 0.27%) due to issues in label generation, calibration data errors, and insufficient training epochs, the project provided deep practical insights into 3D object detection, BEV-based LiDAR processing, and FPGA-based edge AI deployment. The KITTI-pretrained Complex YOLOv4 model (93% accuracy, 3 classes) was successfully deployed on the ZYNQ board, demonstrating the full hardware deployment pipeline.

The challenges encountered — NaN training losses, incorrect calibration matrices, and class imbalance — were thoroughly analyzed and documented, making this project a valuable learning experience in embedded systems, deep learning, and hardware-software co-design.

Future Scope

- Fix the label generation pipeline to correctly handle class imbalance and accurate 3D bounding box creation.
- Correct the Calibration matrix to enable accurate BEV projection.
- Train the model for the recommended ~300 epochs with the corrected dataset to achieve significantly improved mAP.
- Explore model pruning and structured sparsity to reduce computational cost for tighter FPGA resource constraints.
- Upgrade to Zynq UltraScale+ MPSoC for improved DPU performance and support for larger models.
- Integrate real-time camera and LiDAR sensor input on the ZYNQ board for live deployment testing.

References

5. Simon, M., et al. (2019). Complex-YOLO: A Euler-Region-Proposal for Real-Time 3D Object Detection on Point Clouds. ECCV 2018 Workshops.
1. Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934.
2. Geiger, A., Lenz, P., & Urtasun, R. (2012). Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. CVPR 2012.
3. Xilinx/AMD. (2023). Vitis AI User Guide (UG1414). https://docs.xilinx.com/r/en-US/ug1414-vitis-ai

4. Complex-YOLOv4-Pytorch GitHub Repository. https://github.com/maudzung/Complex-YOLOv4-Pytorch
5. LabelImg Annotation Tool. https://github.com/heartexlabs/labelImg
6. PYNQ Framework Documentation. http://www.pynq.io
7. Ultralytics YOLOv5 Documentation. https://docs.ultralytics.com